

SITUAÇÃO-PROBLEMA COM PENSAMENTO COMPUTACIONAL

Semana de Tecnologia — Distribuição de 240 alunos em 8 oficinas

TRABALHO EM DUPLA
Guilherme Dos Santos Tavares Pedro
Aplicação dos quatro pilares: decomposição, reconhecimento de padrões, abstração e algoritmos.

Objetivo do trabalho

Demonstrar como uma situação real de organização escolar pode ser transformada em um processo lógico, estruturado e verificável, utilizando os quatro pilares do pensamento computacional.

1. Apresentação da situação-problema

A escola realizará uma Semana de Tecnologia com oito oficinas acontecendo simultaneamente. Foram inscritas 240 pessoas, e cada estudante poderá participar de apenas uma oficina. Como as inscrições foram feitas por ordem de chegada, os primeiros inscritos possuem prioridade na escolha. Ao mesmo tempo, nenhuma oficina pode ultrapassar sua capacidade e ninguém deve terminar o processo sem uma oficina.

O desafio consiste em criar uma estratégia de distribuição que seja justa, organizada, fácil de conferir e capaz de lidar com oficinas que atinjam sua lotação antes do final das inscrições.

Oficina	Capacidade
Robótica	40
Programação	50
Design de Games	30
Impressão 3D	20
Cibersegurança	25
Inteligência Artificial	35
Edição de Vídeo	20
Redes de Computadores	20
TOTAL	240

1.1. Conferência matemática

$40 + 50 + 30 + 20 + 25 + 35 + 20 + 20 = 240$ vagas. Portanto, a capacidade total é exatamente igual ao número de alunos. No entanto, isso não significa automaticamente que qualquer conjunto de preferências será compatível com as vagas. Para garantir que todos possam ser atendidos, a solução proposta exige que cada estudante classifique as oito oficinas, da primeira à última preferência.

1.2. Objetivo geral

- Distribuir os 240 alunos sem duplicidade.
- Respeitar a ordem de inscrição como critério de prioridade.
- Nunca ultrapassar a capacidade de uma oficina.
- Considerar as preferências de cada estudante.
- Registrar o resultado de forma que possa ser conferido pela coordenação.
- Produzir uma solução que possa ser reutilizada em outros eventos.

2. Pilar 1 — Decomposição

Decomposição é dividir um problema complexo em partes menores e mais fáceis de controlar. Em vez de tentar distribuir 240 alunos de uma só vez, o problema é separado em cadastro, preferências, capacidade, alocação, resolução de conflitos e validação.

Etapas	O que fazer	Resultado esperado
1	Organizar inscrições	Lista única em ordem de chegada.
2	Validar cadastros	240 inscrições válidas e sem duplicidade.
3	Registrar preferências	Ranking das 8 oficinas para cada aluno.

Etapa	O que fazer	Resultado esperado
4	Cadastrar capacidades	Limite correto para cada oficina.
5	Inicializar vagas	Controle das vagas restantes.
6	Alocar alunos	Processamento do primeiro ao último.
7	Resolver conflitos	Buscar a próxima preferência quando necessário.
8	Validar resultado	Conferência de todas as regras.
9	Gerar relatório	Lista final por aluno e por oficina.

2.1. Benefício da decomposição

Essa divisão permite localizar erros rapidamente. Se o total de vagas estiver errado, o problema está no cadastro das oficinas. Se um aluno aparecer duas vezes, o problema está na lista de inscrições. Se uma oficina tiver mais alunos que sua capacidade, o erro está no controle de vagas ou na etapa de alocação.

3. Pilar 2 — Reconhecimento de Padrões

Reconhecer padrões significa perceber comportamentos e regras que se repetem. Neste caso, todos os alunos passam pelo mesmo tipo de análise: suas preferências são consultadas na ordem, e a primeira opção com vaga é escolhida.

Padrão identificado	Como aparece no problema
Ordem de inscrição	O aluno mais antigo é analisado antes dos demais.
Capacidade fixa	Cada oficina possui um limite que não pode ser excedido.
Preferências em sequência	A primeira opção é tentada antes da segunda, terceira e assim por diante.
Lotação	Quando uma oficina chega a zero vagas, ela deixa de ser opção disponível.
Consumo de vaga	Cada aluno atribuído reduz exatamente uma vaga.
Validação repetível	As mesmas verificações podem ser aplicadas em qualquer edição do evento.

3.1. Exemplo de padrão de demanda

Se Programação for escolhida como primeira opção por muitos estudantes, as 50 vagas poderão ser ocupadas rapidamente. O padrão é previsível: enquanto houver vaga, o estudante é alocado; quando a oficina lotar, o próximo estudante segue para sua segunda preferência. Esse mesmo comportamento pode ocorrer com qualquer outra oficina.

3.2. Por que reconhecer padrões?

Porque padrões permitem criar uma regra única em vez de inventar uma decisão diferente para cada aluno. Isso torna o processo mais rápido, objetivo e menos sujeito a decisões arbitrárias.

4. Pilar 3 — Abstração

Abstração significa manter apenas os dados que realmente interferem na solução. A coordenação não precisa considerar informações pessoais que não alteram a distribuição das vagas.

Informação	Usar?	Justificativa
Identificação do aluno	SIM	Permite localizar a pessoa no resultado.
Ordem de inscrição	SIM	Define a prioridade.
Preferências	SIM	Indicam a ordem das escolhas.
Oficina	SIM	Representa o destino possível.
Capacidade	SIM	Impede excesso de alunos.
Vagas restantes	SIM	Mostra a disponibilidade atual.
Oficina atribuída	SIM	Registra o resultado.
Cor da camiseta	NÃO	Não influencia a distribuição.
Aparência física	NÃO	É irrelevante para o problema.
Outros detalhes sem relação	NÃO	Apenas aumentariam a complexidade.

4.1. Modelo abstrato dos dados

Aluno: identificação + posição na fila + preferências + oficina atribuída.

Oficina: nome + capacidade + vagas restantes + quantidade de alunos alocados.

Esse modelo é suficiente para tomar as decisões do problema sem carregar informações desnecessárias.

5. Pilar 4 — Algoritmos

Um algoritmo é uma sequência ordenada de instruções para chegar a um resultado. Aqui, o algoritmo transforma as regras da escola em decisões objetivas e repetíveis.

5.1. Algoritmo completo — passo a passo

1. Receber a lista de inscrições.
2. Conferir se existem 240 alunos válidos.
3. Verificar se não há inscrições duplicadas.
4. Cadastrar as oito oficinas e suas respectivas capacidades.
5. Somar as capacidades e confirmar o total de 240 vagas.
6. Para cada aluno, registrar as oito oficinas em ordem de preferência.
7. Definir, para cada oficina, o número de vagas restantes.
8. Começar pelo primeiro aluno da lista.
9. Consultar a primeira preferência.
10. Se houver vaga, atribuir essa oficina ao aluno.
11. Diminuir em 1 a quantidade de vagas restantes.
12. Se não houver vaga, consultar a próxima preferência.
13. Continuar até encontrar uma oficina disponível.
14. Passar ao próximo aluno somente depois de concluir o anterior.
15. Repetir até o 240º aluno.
16. Executar as validações finais.

17. Gerar a lista de alunos por oficina e o relatório de conferência.

5.2. Regra de decisão

- Se houver vaga: o aluno recebe a oficina daquela preferência.
- Se estiver lotada: a preferência é descartada e o algoritmo passa para a próxima.
- Depois da atribuição: o aluno recebe exatamente uma oficina e não disputa outra.
- Se nenhuma preferência estiver disponível: o sistema deve registrar uma inconsistência para correção; com o ranking completo das oito oficinas e capacidade total de 240, a situação não deve ocorrer.

5.3. Pseudocódigo

INÍCIO

ler alunos em ordem de inscrição

ler oficinas e capacidades

para cada oficina: vagas_restantes = capacidade

para cada aluno, do primeiro ao último:

alocado = falso

para cada preferência do aluno:

se vagas_restantes[preferência] > 0:

atribuir preferência ao aluno

vagas_restantes[preferência] = vagas_restantes[preferência] - 1

alocado = verdadeiro

parar a busca

se alocao = falso:

registrar inconsistência

validar total de alunos

validar uma oficina por aluno

validar limites de capacidade

gerar relatório final

FIM

6. Exemplo prático de funcionamento

Considere três alunos consecutivos. Todos colocaram Programação como primeira opção, mas possuem alternativas diferentes. O algoritmo não muda a ordem de prioridade entre os alunos: ele apenas verifica a disponibilidade no momento em que cada um é processado.

Ordem	Aluno	1ª opção	2ª opção	Decisão
1	Aluno 001	Programação	Robótica	Programação, se houver vaga.
2	Aluno 002	Programação	IA	Programação, se houver vaga.
3	Aluno 003	Programação	Robótica	Se Programação estiver lotada, Robótica.

6.1. Quando uma oficina lota?

- As vagas restantes chegam a zero.
- A oficina passa a ser considerada indisponível.
- Nenhum aluno posterior pode ultrapassar o limite.
- O próximo aluno tenta a próxima preferência.
- A ordem de inscrição continua sendo respeitada.

7. Validação e controle de qualidade

Uma solução computacional não termina quando os alunos são distribuídos. É necessário verificar se o resultado realmente obedece às regras. A validação reduz o risco de erros antes da publicação da lista

final.

Regra	Teste de validação	Resultado esperado
Quantidade de alunos	Contar registros finais	240
Uma oficina por aluno	Contar atribuições por aluno	Exatamente 1
Capacidade	Comparar ocupação e limite	Ocupação $\leq$ capacidade
Prioridade	Conferir sequência de processamento	Ordem original preservada
Total distribuído	Somar alunos de todas as oficinas	240
Vagas restantes	Somar vagas não utilizadas	0 ao final, neste cenário
Duplicidade	Procurar aluno em mais de uma oficina	Nenhum duplicado

7.1. Checklist final

- 240 inscrições válidas.
- Lista em ordem de chegada.
- Oito oficinas cadastradas.
- Capacidade total igual a 240.
- Preferências registradas para todos.
- Nenhum aluno duplicado.
- Nenhum aluno em duas oficinas.
- Nenhuma oficina acima da capacidade.
- 240 alunos alocados.
- Relatório final conferido.

8. Fluxo resumido da solução

Inscrições → Validação → Preferências → Cadastro das vagas → Processamento por ordem → Verificação de disponibilidade → Atribuição → Atualização das vagas → Validação final → Relatório.

8.1. Entrada, processamento e saída

Entrada	Processamento	Saída
Lista de alunos	Ordenar e validar	Lista válida
Preferências	Buscar primeira opção disponível	Oficina atribuída
Capacidades	Controlar vagas restantes	Ocupação por oficina
Regras	Executar validações	Relatório de conformidade

8.2. Possíveis formas de implementação

A mesma lógica pode ser implementada manualmente com uma planilha eletrônica, em um programa simples ou em um sistema de inscrições. A estrutura dos dados e as regras permanecem iguais; o que muda é apenas a ferramenta usada para executar o algoritmo.

9. Eficiência da solução

Se existem N alunos e cada um apresenta até M preferências, o algoritmo pode realizar, no pior caso, aproximadamente $N \times M$ verificações. Neste problema, $N = 240$ e $M = 8$, portanto o limite aproximado é de 1.920 verificações de preferência. Para um computador, é uma quantidade pequena e facilmente processável.

Além disso, o controle de vagas é simples: cada atribuição reduz uma unidade da oficina escolhida. Isso evita a necessidade de recalcular toda a distribuição a cada passo.

10. Ponto crítico — como garantir que ninguém fique sem oficina

Existe uma diferença importante entre capacidade total e compatibilidade das preferências. A escola possui exatamente 240 vagas para 240 alunos, mas isso só garante o atendimento de todos se cada aluno puder recorrer a alternativas quando sua primeira escolha estiver lotada.

Por isso, a melhor regra de cadastro é solicitar que cada estudante ordene todas as oito oficinas. Dessa maneira, o algoritmo possui um caminho de decisão completo: se a primeira estiver cheia, tenta a segunda; depois a terceira; e assim por diante.

Como o total de vagas é exatamente 240 e cada estudante ocupa somente uma vaga, quando os 240 alunos forem processados, todas as vagas disponíveis terão sido utilizadas. A prioridade continua sendo a ordem de inscrição, e as capacidades individuais continuam sendo respeitadas.

11. Justiça, transparência e auditoria

A solução evita escolhas subjetivas porque usa critérios definidos antes da distribuição. Todos os estudantes são submetidos à mesma regra, e a ordem de chegada é preservada.

- Critério objetivo: ordem de inscrição.
- Critério de escolha: preferência disponível de maior prioridade.
- Limite objetivo: capacidade cadastrada de cada oficina.
- Registro: cada aluno recebe uma atribuição identificável.
- Auditoria: o resultado pode ser conferido comparando alunos, oficinas e capacidades.

Em caso de contestação, a coordenação pode consultar a posição de inscrição, as preferências registradas e o momento em que cada oficina ficou sem vagas. Isso torna a decisão explicável e verificável.

12. Solução reutilizável para futuros eventos

A proposta não depende exclusivamente dos números deste evento. Em outra edição, a escola poderá ter mais ou menos alunos, novas oficinas, diferentes capacidades e outra quantidade de preferências. Basta atualizar os dados de entrada e manter as regras do algoritmo.

Elemento variável	Pode mudar?	Exemplo
Número de alunos	Sim	180, 240, 300...
Nome das oficinas	Sim	Novos temas ou atividades.
Capacidade	Sim	Laboratório maior ou menor.
Preferências	Sim	Ranking parcial ou completo, conforme a regra.
Ordem de prioridade	Sim	Ordem de inscrição ou outro critério definido previamente.

12.1. Vantagens

- Reduz erros de contagem.
- Mantém o critério de prioridade de forma objetiva.
- Evita ultrapassar vagas.
- Facilita a conferência pela coordenação.
- Pode ser automatizada.

- Pode ser adaptada a eventos futuros.
- Permite gerar relatórios para alunos e gestores.

13. Resumo dos quatro pilares

Pilar	Aplicação no problema
Decomposição	Dividir cadastro, preferências, vagas, alocação, conflitos e validação.
Reconhecimento de padrões	Perceber que todos os alunos passam pela mesma sequência de verificações.
Abstração	Manter somente identificação, ordem, preferências, oficinas e capacidades.
Algoritmos	Executar regras passo a passo até produzir a distribuição final.

14. Conclusão

A distribuição dos 240 alunos entre as oito oficinas é um problema de organização que pode ser resolvido de maneira estruturada com o pensamento computacional. A decomposição divide o desafio em partes menores; o reconhecimento de padrões identifica regras repetidas; a abstração elimina informações desnecessárias; e o algoritmo transforma as regras em uma sequência precisa de ações.

A solução proposta preserva a ordem de inscrição, respeita os limites das oficinas e garante que cada aluno receba apenas uma oficina. O ponto essencial para tornar a solução robusta é permitir que cada estudante ordene as oito oficinas por preferência. Assim, quando uma escolha estiver lotada, o sistema poderá buscar automaticamente a próxima alternativa.

Dessa forma, a proposta não apenas resolve a situação apresentada, mas também demonstra como o pensamento computacional pode transformar um problema real em um processo lógico, transparente, eficiente e reutilizável.

Integrantes: Guilherme Dos Santos Tavares e Pedro